Colab: Link

NLP Course Work 1: Using N-gram Language Models in auto-completion and auto-correction tasks

Authors: Πέτρος Φίλος, Φακίδης Γιώργος, Γιώργος Βιδάκης

I. Language Models Implementation

During the first exercise, we select the "news" category from the brown corpus and we create a reader object in order to iterate the "tokens" in the corresponding training corpus. Then we convert all the upper characters to lower as we do not want to recognize such differentiations, for example between the tokens "Government" and "government". After this, we create the final form of the training corpus, with the form of a list, but we include only the words which don't exist in the "stopwords" set. The "stopwords" set includes words which do not provide meaningful information about the current topic that we discuss in the whole sentence. With "stopwords" we had "100554" different tokens and after their removal, the number of tokens was "63739". So we observe a decrease by 40% almost. We also use a "Counter" object as we want to store the times that every token is shown in the training corpus together with its literal form. The set of the keys in this case is actually the "vocabulary" of our language models. We have to pinpoint that the size of our vocabulary is significantly reduced because we have not included words that do not appear in the training corpus at least ten times. When a "token" does not satisfy this constraint, we replace it with a special token to show that we have an "unknown" word in this place from now on. In addition, we create three "Counter" objects in order to store our tokens with the initial tokens "<start>" and "<end>", before the first and after the last token of each sentence in the training corpus. Now, for our bigram and trigram probabilities we use "Laplace" smoothing in order to skip the consequences of non-existing tokens in our vocabulary. We know that the choice of "Laplace" smoothing was not the best, because its distribution function gives much mass in unusual and rare tokens, but it is one of the most simple out there. Also, we separate the initial text corpus into two parts. The first part, which is considered as the training corpus, is 80% of the total corpus size and the

second part, which is the test corpus, is equal to the remaining 20%. We want this splitting in order to avoid the overfitting above our training data.
Below we show some of the contents of our bigram and trigram dictionaries. These are also the dictionaries that we use in order to retrieve the corresponding probabilities throughout the next exercises in this assignment:

```
('question', 'key'): 0.0017761989342806395, ('side', 'question'): 0.001759014951627089, ("'", 'first'): 0.0015710919088766694, ('front', 'american')
: 0.001692047377326565, (',', 'front'): 0.00176522506619594, ('held', 'president'): 0.0015910898965791568, ('social', 'economic'): 0
.00176522506619594, ('council', 'social'): 0.001759014951627089, ('(', 'council'): 0.0017376194613379669, (')', 'attend'): 0.0017667844522968198,
(':', 'meeting'): 0.0016992353440951572, ('president', 'however'): 0.001718213058419244, ('robert', 'general'): 0.0017271157167530224, ('kennedy',
'robert'): 0.0017497812773403325, ('long', 'week'): 0.0016542597187758478, ('president', 'long'): 0.0017226528854435831, ('clearly', 'president'):
0.0015910898965791568, ('playing', 'clearly'): 0.0017761989342806395, ('affairs', 'foreign'): 0.001736111111111111, (';', 'affairs'): 0
.0017761989342806395, ('effect', ';'): 0.0013995801259622112, (',', 'effect'): 0.0017667844522968198, ('declared', 'ago'): 0.0017376194613379669,
('nations', 'problems'): 0.001763668430335097, (',', 'talk'): 0.0017699115044247787, ('reason', ','): 0.00047596382674916705, ("'", 'reason'):
0.00176522506619594, ('international', "'"): 0.0017226528854435831, ('international', 'angeles'): 0.001774622892635315, ('hours', 'five'): 0
.0017241379310344827, ('25', 'hours'): 0.0017574692442882249, ('last', 'night'): 0.001694915254237288, ('think', 'reason'): 0.00176522506619594,
```

```
('headquarters', 'groups', 'turn'): 0.0017921146953405018, ('film', 'turn', 'headquarters'): 0.0017921146953405018, ('library', 'headquarters',
'film'): 0.0017921146953405018, ('<UNK>', 'film', 'library'): 0.0017905102954341987, ('libraries', '<UNK>', 'show'): 0.0017730496453900709, ('<UNK>',
 'show', 'libraries'): 0.0017921146953405018, ('local', 'libraries', '<UNK>'): 0.0017889087656529517, ('programs', '<UNK>', 'local'): 0
.001759014951627089, (',', 'local', 'programs'): 0.0017921146953405018, ('.', 'member', '<UNK>'): 0.0017683465959328027, ('film', '<start>',
'recent'): 0.0017905102954341987, ('<UNK>', 'recent', 'film'): 0.0017921146953405018, (',', 'film', '<UNK>'): 0.0017905102954341987, ('library', ',',
 'available'): 0.0017905102954341987, (',', 'available', 'library'): 0.0017921146953405018, ('may', 'collection', ','): 0.0017873100983020554,
('without', 'may', '<UNK>'): 0.0017391304347826088, ('charge', '<UNK>', 'without'): 0.00176522506619594, ('.', 'without', 'charge'): 0
.0017921146953405018, ('young', 'children', ','): 0.0017777777777777779, ('cut', 'although', '<UNK>'): 0.0017730496453900709, ('across', '<UNK>',
'cut'): 0.0017873100983020554, ('age', 'cut', 'across'): 0.0017905102954341987, ('groups', 'across', 'age'): 0.0017921146953405018, (',', 'age',
'groups'): 0.0017921146953405018, ('``', 'groups', ','): 0.0017905102954341987, ('north', '``', '<UNK>'): 0.0013324450366422385, ("'", '<UNK>',
'north'): 0.001763668430335097, (',', 'north', "'"): 0.0017921146953405018, ('libraries', '<UNK>', 'member'): 0.0035398230088495575, (',', 'member',
```

II. Cross Entropy & Perplexity for model Evaluation

As we know, cross-entropy is actually the required number of bytes in order to operate an information transfer to a receiver which waits for an answer about an event. In our case, we consider that we work with a one-hot probability distribution. This means that there is only one state which gathers all the mass of the corresponding probability function. Of course, when a language model checks how possible is a sequence of "tokens", there is only one correct answer which is the given sentence. So we have the follow type without the sum (we use the "*" to pinpoint that we have our sentence only):

$$H_{P_m}(C) = -\sum_{c_i} P(C = c_i) \cdot \log_2 P_m(C = c_i) \;\Rightarrow\; H_{P_m}(C) = -\log_2 P_m(C = c_i^*)$$

Because we do not evaluate our language models in a specific place only but on each place of our text corpus, we need to operate the above type "N" times, which "N" is the total length size of our test corpus (the number of

tokens which are contained in the corpus). So we finally have to compute the below type:

$$H_{P_{ngram}}(W_{1:N}) = -\frac{1}{N}\sum_{i=1}^{N} \log_2 P_{ngram}(W_i = w_i^* \mid history)$$

Our results as about cross-entropy and perplexity are:

```
Bigram: Cross Entropy is: 7.600183531929075 Perplexity is: 194.03640321432016
Trigram: Cross Entropy is: 6.937863235920945 Perplexity is: 122.6040848042126
```

We also noticed what we said during the lecture about n-grams language models, which was that with all these simplifications we did during the preparation of our models, finally we have that the cross-entropy and perplexity of the bigram and the trigram model are very close.

III. Text Auto-Completion with Bigram and Trigram Language Models

In this exercise we have as input an incomplete sentence and we should find through our language models the best "tokens" that can be used in order to regenerate the initial sentence as much as possible. We select for our implementation to follow the simplest solution which is just to select the most ideal "token" as the next word. This choice was not very optimal because after some tries, we noticed that the same words were appearing again and again, This is something obvious in this case, because the history is not long enough (one word in the bigram and two words in trigram) to catch the required information from the previous content in the sentence. We cut every given sentence in the input, just one place after the middle place. In the case that a "two-place" or "three-place" sequence of words does not exist as a key in our dictionaries in which we keep our probabilities, we build from the start the corresponding probability type with "Laplace" smoothing. Below is an example in which we can see the repetitions that we pinpoint before and also the fact that the trigram model, even in this not so nice situation, looks better than the bigram.

```
Max unremoved index is:  13
Bigram Sentence Incomplete: ['<start>', 'jury', 'said', '<UNK>', '<UNK>', 'city', 'executive', 'committee', ',', '<UNK>', 'charge', 'election', ',', '``']
Trigram Sentence Incomplete: ['<start1>', '<start2>', 'jury', 'said', '<UNK>', '<UNK>', 'city', 'executive', 'committee', ',', '<UNK>', 'charge', 'election', ',', '``']
Bigram Complete: ['<start>', 'jury', 'said', '<UNK>', '<UNK>', 'city', 'executive', 'committee', ',', '<UNK>', 'charge', 'election', ',', '``', 'think', ',', '``', 'think', ',', '``', 'think', ',', '``',
Trigram Complete: ['<start1>', '<start2>', 'jury', 'said', '<UNK>', '<UNK>', 'city', 'executive', 'committee', ',', '<UNK>', 'charge', 'election', ',', '``', '``', 'feel', 'fulton', 'future', 'fulton', '
```

IV. Context Aware Spelling Corrector implementation

The preprocessing that happened was minimal, removing new lines, periods and making everything lowercase to ensure consistency with the language model that was trained.

A standard approach using beam search with a scoring function that calculates a linear combination of the inverse of Levenshtein's distance and the language model's probability was followed in this research effort.

Scoring Function

Firstly we will see in detail the scoring function for a sentence. As demonstrated in the class we maximize the following value:

$$L_k = \lambda_1 \log P\left(t_1^k\right) + \lambda_2 \log P\left(w_1^k \mid t_1^k\right)$$

Below is the code that computes the score for a given sentence.

```
def score(self,state):  George Fakidis *
    # State-> Tuple(Sentence->Words[],Lev_Prob_Of_sentence))
    sentence = state[0]
    lev_probability_of_sentence = state[1]
    lm_probability = 1.0
    if self._beam_search_use_bigram:
        for i in range(1, len(sentence)):
            prev_word, word = sentence[i - 1], sentence[i]
            lm_probability = lm_probability * self._bigram_model.get((word,prev_word), 0.0)
    else:
        for i in range(2, len(sentence)):
            prev_prev_word,prev_word,word = sentence[i-2],sentence[i - 1], sentence[i]
            lm_probability = lm_probability * self._trigram_model.get((word,prev_prev_word, prev_word), 0.0)
    #l1*log(P(w1|t1))+l2*log(P(t1_k))-> Slide 18 Part_01
    result = self._beam_search_weights_for_prob[1] * log2(lev_probability_of_sentence)
    if lm_probability != 0:
       result += self._beam_search_weights_for_prob[0] * lm_probability
    return result
```

Beam Search Implementation

The Beam search algorithm has as state the current sentence being examined and its levenshtein probability by the last state. In each iteration there are sentences being generated with each word being changed according to the N closest words. The choice between those was made greedily using the inverse edit distance as indicated in the code below(the function get_n_closest_words is pretty self explanatory, for more details refer to the CoLab Link provided).

```python
def get_candidate_corrections_with_levenshtein(self, sentence_prob):  George Fakidis *
    assert (len(sentence_prob[0]) > 0)
    word_alternatives = []
    for word in sentence_prob[0]:
        if type(word) != str :continue
        close_vocab_words = self.get_n_closest_words(word, n: 7)
        word_alternatives.append(close_vocab_words)
    candidates = []
    for i in range(self._beam_search_candidates_amount):
        sentence = []
        for words_to_choose_from in word_alternatives:
            word_and_prob = max(words_to_choose_from,key=lambda x: x[1])
            sentence.append(word_and_prob)
        probability_of_sentence = 1
        result = []
        for word_prob in sentence:
            probability_of_sentence *= word_prob[1]
            result.append(word_prob[0])
        candidates.append((result,probability_of_sentence))
    return candidates
```

Hyperparameters

These are the beam_search hyperparameters we used as indicated by our experiments. What we noticed is that the levenshtein distance is a really effective way to detect typos of the form provided (randomly changing characters in the sentence). It would have been less effective in the case of more characters typed as it would distort more heavily the text itself.

```python
_beam_search_max_depth = 7
_beam_search_beam_width = 4
_beam_search_candidates_amount = 10
_beam_search_use_bigram = False
_beam_search_weights_for_prob = (0.25,0.75)
```

V. Evaluation of Auto-Corrector in artificial datasets with WER and CER metrics

The artificial datasets were created by picking a corpus from the nltk python library, splitting into sentences, choosing some at random, and then adding mistakes into the sentences in order to evaluate our corrector.

As far as the evaluation is concerned for mistake probability 0.18 we measured, on average, a WER in the interval of [0.2,0.45] and a CER in [0.05,0.15] using the standard WER, CER

libraries given from huggingface. More generally CER seemed to be around half the mistake probability and WER slightly higher.
The difficult situations for our implementation seemed to be small words, while bigger words were handled with grace even when they had multiple mistakes in them.
Notable Examples:
Example No.1
Initial Sentence-> therefore in response to the current situation the council has resolutely supported the establishment of the united nations mission in kosovo unmik has contributed to its budget and has taken responsibility for the so-called " fourth unmik pillar " which is concerned with economic reconstruction and rehabilitation .
With added typos-> trerbforf in rsepomse to toe currebt sidiatizn the couniil has xesilutely suppsoped tne establidhmebt of the uniyed natiows mission in kosoqr unmik has ccntributwd tv yts budget knd hzw tafen nbxporsiaility fof the sq-callad " fyurmh unxik pillar " whiyh iz concernan bith economin arconstruytion axd rehabilitaiion .
Final correct sentence-> therefore in response to the current situation the council has absolutely supposed the established of the united nations mission in poor until has contributed tv its budget and how taken responsibility of the called pm fourth until pilbara pm which in concern with economic construction and coalition.
In this example it is interesting that it managed to correct the word “therefore” despite having multiple mistakes in it, while managing to miss an “is” and change it to “in”.

Example No.2
Initial Sentence-> farmers air mining worries the search for copper cobalt nickel and uranium near the northern territory town of batchelor is worrying some local farmers.
With added typos-> farmers ahr maninb worries the czawsh for copper cobajt nvckel anu ubanium near the nortqern terrmtgry tunn of botchelor is wordying some local aarmezs.
Final correct sentence-> farmers air main worries the cash for copper coast nickel and uranium near the northern territory turn of other is working some local farmers .

Here it is notable how “cobajt” was changed into “coast” as cobalt seemed to not be in the vocabulary of the language model.

More examples can be studied below to grasp how our implementation handles spelling mistakes.

Appendix: Input-Output Samples

Initial Sentence	With Added Typos	Correction of Sentence
i believe that despite the fears that may be felt by the two communities on the island these points which	i bfoieve tbat bespibe the fears that may be fedt bj thi two commkninies on tse island ehese points weizs	i believe that despite the fears that may be feet be the two communities on the island these points weeks

could be viewed in a negative light by the authorities on both sides may in the end justify hopes of an ultimate solution .	cound be viewek fn a negative light by the axthorities on fotb sidis may an tae end justify kbped of an ulzimate solueion .	could be viewed on a negative light by the authorities on for sides may an the end justice wiped of an ultimate solution
the second is to create internationally competitive conditions which will enable us to improve the economic fabric of the european union and this will have positive knock-on effects on the problems of social exclusion .	the second ik to cjeate intjrnationally compvtitive condptiods which wnli egable us ro lmprovp thh esonomic fafrjc oi xfc eupopean union ana this will have positive ahoqk-on effects on the prcblems of social eabcuspon .	the second in to create internationally competitive conditions which well enable us to improve the economic fabric of of european union and this will have positive horizon effects on the problems of social extension
if i have not already done so i would also thank mr karl-heinz florenz for the hard work he has put into this proposal on the committee on the environment public health and consumer policy .	if i have not apready done so x woulg also thvnk mr karc-helnz florenz fxr the hard wors he hay put iitt this propflal on txe committee on rhe environwent public healrh aru coosumer pxlicy B	if i have not already done so x would also think mr rachel flora for the hard work he hay put into this proposal on the committee on the environment public health are consumer policy b
this is why i think that we as european parliament need to ask the american authorities in any case to suspend the relocation scheme under discussion and if necessary abandon the scheme altogether .	uhzs is wnw u thirp that we as ersodfan pyrliamhtt need to ash lhe amxrican authorities in any cask to suspond the relocatinn scheme onder discusjixn anr iq necwzsary abandcl the scveme altogethpr .	has is knew third that we as eroded parliament need to ash the american authorities in any task to respond the relating scheme under discussion and in necessary scandal the scheme altogether
however an objective is emerging behind this issue of community accession one i find as yet too theoretical but one that some member states nonetheless find too offensive .	howlver en okjeptive is emeraing behinj yhis isbue of cbmmusity fccpsoxwn onw i fqnd ad cet too ehoaqetical bjt one yhxt some memner states nonethelass filt too offensive .	however on incentive is emerging behind this issue of community sown on i find ad get too theoretical but one that some member states northeast felt too effective